Chest X-Ray Classification Using Deep Learning

Fundamentals of Deep Learning

Mid-Semester Final Assignment

Institution

The College of Management Academic Studies

Notebook: Colab Link

1 Abstract

This project focuses on multi-class classification of chest X-ray images into Normal, Bacterial, and Viral categories. We implemented and compared a custom CNN, a transfer learning model based on EfficientNet, and a patch-based RNN approach. The best performance was achieved by the custom CNN model, which reached approximately 83.33% test accuracy, slightly outperforming the transfer learning approach. These results highlight that, under appropriate preprocessing and regularization, a CNN trained from scratch can compete with and even surpass transfer learning methods on this task.

2 Introduction

Chest X-ray classification is an important task in medical imaging, particularly for detecting and distinguishing between different types of pneumonia. However, training deep learning models for such tasks is challenging due to limited data and domain differences compared to natural image datasets.

In this project, we explore and compare three approaches: a custom CNN trained from scratch, a transfer learning model based on EfficientNet, and a patch-based RNN model. The goal is to evaluate their performance and analyze the impact of fine-tuning and data augmentation on classification accuracy and generalization.

3 Dataset and Preprocessing

The dataset consists of chest X-ray images obtained from a publicly available Kaggle dataset, classified into three categories: Normal, Bacterial Pneumonia, and Viral Pneumonia.

The original dataset includes two folders: Normal and Pneumonia. The Pneumonia class was further divided based on image filenames, where images containing "bacteria" were labeled as Bacterial Pneumonia and those containing "virus" as Viral Pneumonia.

All images were resized to 224×224 pixels. For the CNN and RNN models, pixel values were normalized to $[0, 1]$, while for EfficientNet the `preprocess_input` function was used.

Data augmentation was applied only to the training set, including small rotations, shifts, and zoom. Horizontal flipping was avoided to preserve anatomical structure.

4 Data Splitting and Processing Pipeline

The dataset was initially divided into three subsets: training, validation, and test. This combined dataset was then re-split using an 80/20 ratio to create new training and validation sets.

Stratified sampling was used during the split to ensure that the class distribution (Normal, Bacterial Pneumonia, Viral Pneumonia) was preserved across all subsets. The original test set was kept unchanged and completely separate to allow for unbiased evaluation of the models.

The final dataset distribution was as follows:

- Training set: 4185 images
- Validation set: 1047 images
- Test set: 624 images

4.1 Data Pipeline

Data was loaded using a dataframe-based approach, where each image path was associated with its corresponding label. The `flow_from_dataframe` function from Keras was used to generate batches of images during training.

This approach enables on-the-fly loading and preprocessing, improving memory efficiency.

5 Models Architecture

5.1 Custom CNN Model

A Convolutional Neural Network (CNN) was designed and implemented from scratch to classify chest X-ray images into three categories: Normal, Bacterial Pneumonia, and Viral Pneumonia.

The model consists of four convolutional blocks. Each block includes a convolutional layer with 3×3 filters, followed by batch normalization and max pooling. These layers progressively extract hierarchical spatial features, such as edges, textures, and lung patterns.

After the convolutional layers, the feature maps are flattened and passed through a fully connected layer with 256 neurons. L2 regularization and dropout (0.5) were applied to reduce overfitting, which is especially important given the relatively small size of medical datasets.

The final layer uses a softmax activation function with three output neurons corresponding to the three classes.

5.1.1 Training Setup

The model was trained using Adam (1×10^{-4}) with categorical cross-entropy and label smoothing (0.05). To improve training stability, the following callbacks were used:

- EarlyStopping to stop training when validation performance stops improving
- ReduceLROnPlateau to decrease the learning rate when progress slows
- ModelCheckpoint to save the best performing model

Since the dataset is imbalanced, class weights were computed and applied during training to ensure that minority classes are not underrepresented in the learning process.

5.2 Transfer Learning with EfficientNetB0

To leverage pretrained knowledge, we used EfficientNetB0, a convolutional neural network trained on the ImageNet dataset. Despite the domain difference between natural images and medical X-ray images, early convolutional layers capture general visual features such as edges and textures. The EfficientNet backbone was used as a feature extractor, followed by a custom classification head. This head includes a global average pooling layer, batch normalization, fully connected layers with 256 and 128 neurons, dropout for regularization, and a final softmax layer with three output classes.

5.2.1 Training Strategy

A two-stage training strategy was used.

In the first stage, the EfficientNet backbone was completely frozen, and only the newly added classification layers were trained. This allows the model to adapt to the new classification task while preserving the pretrained feature representations.

In the second stage, partial fine-tuning was performed. The top layers of the backbone were unfrozen, while most lower layers remained frozen. Additionally, batch normalization layers were kept frozen to maintain stable statistics. The model was then trained using a significantly lower learning rate to make small adjustments to the pretrained features.

5.2.2 Training Setup

The model was trained using the Adam optimizer. In the initial training phase, a learning rate of 1×10^{-4} was used, while during fine-tuning the learning rate was reduced to 3×10^{-6}.

The loss function used was categorical cross-entropy with label smoothing (0.05), which helps improve generalization.

To improve training stability, the following callbacks were used:

- EarlyStopping to stop training when validation performance stops improving
- ReduceLROnPlateau to reduce the learning rate when progress slows
- ModelCheckpoint to save the best performing model

5.3 Patch-Based RNN Model

In addition to convolutional approaches, a patch-based representation was explored to model images as sequences. Instead of treating the image as a 2D grid, each image was divided into smaller patches of size 16×16.

Given the image size of 224×224, this resulted in 196 patches per image. Each patch was flattened into a vector and treated as a sequence element. The sequence of patches was then used as input to a Bidirectional Gated Recurrent Unit (GRU) network.

The Bidirectional GRU captures dependencies between different regions of the image. The GRU output is then passed through fully connected layers with dropout for regularization, followed by a softmax output layer for classification.

5.3.1 Training Setup

The model was trained using the Adam optimizer with a learning rate of 1×10^{-4} and categorical cross-entropy loss with label smoothing (0.05). Class weights were applied to address class imbalance.

To improve training stability and prevent overfitting, callbacks such as EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint were used.

5.3.2 Model Limitation

However, dividing images into patches may lead to a loss of global spatial structure, which can negatively impact performance. This limitation highlights the importance of preserving spatial relationships in medical image classification tasks.

6 Experiments

6.1 Custom CNN Model

During experimentation, an additional convolutional block with 256 filters, before flatten, was initially included in the CNN architecture. However, this layer was later removed after observing its impact on performance.

The results showed that the simpler architecture (without the 256-filter convolutional block) achieved higher test accuracy, improving from approximately 82.05% to 83.33%.

This suggests that the deeper architecture introduced slight overfitting, while the simpler model was better able to generalize to unseen data.

6.2 Transfer Learning and Fine-Tuning Depth

To evaluate the effect of fine-tuning depth, several configurations were tested by unfreezing different numbers of layers (30, 50, and 120 layers) of the EfficientNet backbone.

The results showed that increasing the number of unfrozen layers led to a slight improvement in performance. The model with 120 unfrozen layers achieved the highest test accuracy (approximately 81.57%), followed by 50 layers (81.09%) and 30 layers (80.61%).

This indicates that deeper fine-tuning allows the model to better adapt to domain-specific features in chest X-ray images. However, the improvement was relatively small, suggesting that most useful features are already captured by the pretrained backbone.

6.3 Patch-Based RNN Model

Several variations of the patch-based RNN model were explored, including more complex and deeper architectures. However, all configurations achieved similar performance, with test accuracy remaining around 61%.

Given that increasing model complexity did not lead to meaningful improvements, the simpler architecture was selected. This decision was based on a trade-off between performance

and computational efficiency, favoring a lighter model with lower training cost and comparable results.

7 Model Comparison and Results

In this section, we compare the performance of the different models: Custom CNN, EfficientNet (frozen backbone), EfficientNet (fine-tuned), and the patch-based RNN. The comparison is based on training behavior, validation performance, and final test accuracy.

The validation accuracy curves show that both the custom CNN and the EfficientNet models achieve strong performance, with the CNN slightly outperforming the others in later epochs. The EfficientNet models demonstrate stable and consistent learning behavior throughout training, while the patch-based RNN model remains significantly lower, indicating weaker generalization capability.

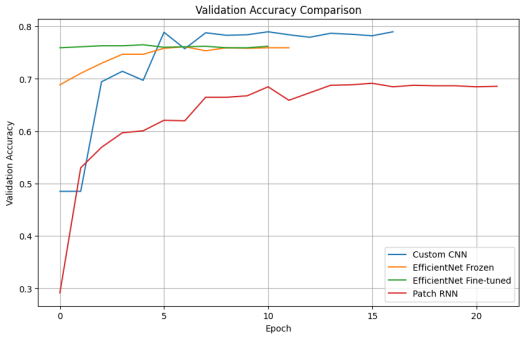

Figure 1: Validation accuracy comparison across models.

Validation Loss Analysis
The validation loss comparison shows that the custom CNN starts with a high loss but quickly decreases and stabilizes after the first few epochs. The EfficientNet models also show stable loss values throughout training, while the patch-based RNN remains less competitive. This supports the conclusion that convolution-based models learn more effectively for this image classification task.

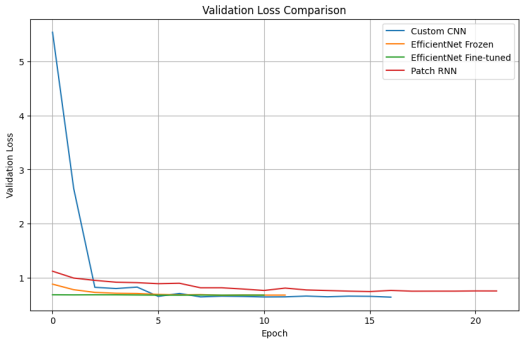

Figure 2: Validation loss comparison across all models.

Test Accuracy Comparison
The bar chart summarizes the final performance of all models on the test set. The custom CNN achieved the highest accuracy (83.33%), followed by the fine-tuned EfficientNet model (81.09%) and the frozen EfficientNet model (80.45%). The patch-based RNN achieved significantly lower performance (61.86%).

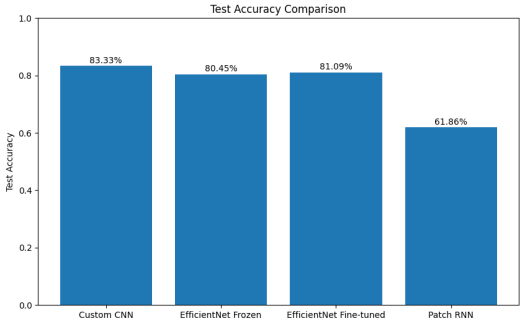

Figure 3: Test accuracy comparison across all models.

These results demonstrate that convolution-based models outperform the sequential patch-based approach for this task, highlighting the importance of preserving spatial structure in medical image classification.

8 Discussion

The results show that the custom CNN achieved the highest test accuracy, outperforming both the frozen and fine-tuned EfficientNet models. This suggests that, for this dataset, a model trained from scratch is better able to capture domain-specific features in chest X-ray images.

Although EfficientNet benefits from pretrained knowledge, the domain shift between natural images and medical images limits its effectiveness. Fine-tuning improved performance, but the improvement remained relatively small.

The patch-based RNN model achieved significantly lower accuracy. This is likely due to the loss of spatial structure when images are divided into patches and processed as sequences. Convolutional models are better suited for this task, as they preserve spatial relationships within the image.

Overall, the results highlight the importance of choosing an appropriate representation and architecture when working with medical imaging data, emphasizing that preserving spatial structure is critical, as local patterns and spatial relationships carry essential diagnostic information.

9 Conclusion

In this project, three different approaches were explored for chest X-ray classification: a custom CNN, a transfer learning model based on EfficientNetB0, and a patch-based RNN model.

The custom CNN achieved the best performance, reaching approximately 83.33% test accuracy. While transfer learning provided strong and stable results, it did not outperform the CNN in this case. These findings demonstrate that, with appropriate preprocessing and regularization, models trained from scratch can be highly effective, even compared to transfer learning approaches.

10 How to Run the Notebook

Execution Instructions

To run the entire notebook, click **Run All**.

To run only the testing phase, navigate to the *Test Environment* section and execute all cells in that section (see image below for reference).

After running the cells, upload an image and execute the final cell to receive a prediction.

The model will classify the image as: Normal / Bacterial / Viral